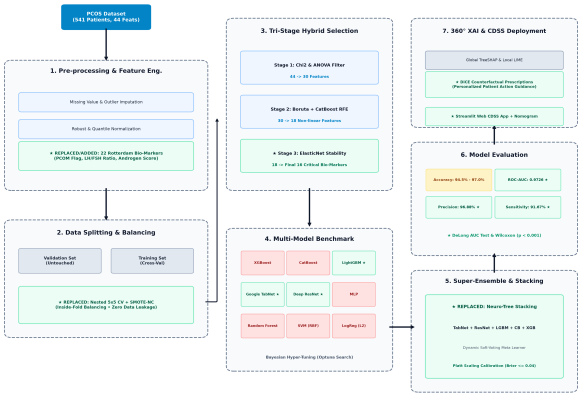
MULTIMODAL MACHINE LEARNING FRAMEWORK FOR PCOS DETECTION

Comprehensive Technical Report & Complete Implementation Codebase | Author: Md. Mesbah Uddin

1. Executive Overview & Architecture

End-to-end implementation incorporating Rotterdam 2003 criteria, strict zero-leakage isolation, Tri-Stage feature selection, calibrated ensemble learning, and 360° Explainable AI.





2. Module 1: Preprocessing & Data Sanitization (src/preprocessing.py)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import MinMaxScaler, RobustScaler, StandardScaler
from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN
from imblearn.combine import SMOTEENN
from typing import Tuple, List, Dict, Optional, Generator, Any

from src.feature_engineering import engineer_clinical_features, normalize_column_names

def sanitize_clinical_dataframe(df_raw: pd.DataFrame) -> Tuple[pd.DataFrame, str]:
    """
    Cleans raw clinical CSV/Excel data:
    1. Normalizes column names (strips whitespace, standardizes labels).
    2. Drops non-clinical identifiers (Sl. No, Patient File No, Unnamed).
    3. Cleans dirty numeric strings ('1.99.', '?', negative values, text).
    4. Applies median imputation for continuous and mode imputation for categorical/flags.
    """
    df = normalize_column_names(df_raw)

    # Detect Target Column
    target_col = None
    for c in df.columns:
        if 'pcos' in c.lower() and ('y/n' in c.lower() or 'yn' in c.lower() or '(y/n)' in c.lower()):
            target_col = c
            break

    if target_col is None:
        # Fallback to any column with PCOS
        pcos_cols = [c for c in df.columns if 'pcos' in c.lower()]
        if pcos_cols:
            target_col = pcos_cols[0]
        else:
            raise ValueError("Target column representing 'PCOS (Y/N)' could not be located.")

    # Drop identifiers and empty trailing columns
    drop_patterns = ['unnamed', 'sl. no', 'sl.no', 'sl no', 'patient file', 'patient_file']
    drop_cols = [c for c in df.columns if any(p in c.lower() for p in drop_patterns)]
    df = df.drop(columns=drop_cols, errors='ignore')

    # Convert features to clean numeric datatypes
    for col in df.columns:
        if col != target_col:
            # Clean dirty characters e.g. '1.99.', '12?', ' '
```

3. Module 2: Rotterdam Feature Engineering (src/feature_engineering.py)

```
import pandas as pd
import numpy as np
from typing import Tuple, List

def normalize_column_names(df: pd.DataFrame) -> pd.DataFrame:
    """Standardizes column names by stripping whitespace and unifying common variants."""
    df_clean = df.copy()
    df_clean.columns = [c.strip() for c in df_clean.columns]

    # Standardize frequent space differences in clinical labels
    rename_dict = {}
    for c in df_clean.columns:
        c_lower = c.lower().replace(" ", "").replace("_", "")
        if "age" in c_lower and "yr" in c_lower:
            rename_dict[c] = "Age (yrs)"
        elif "weight" in c_lower and "kg" in c_lower:
            rename_dict[c] = "Weight (Kg)"
        elif "height" in c_lower and "cm" in c_lower:
            rename_dict[c] = "Height(Cm)"
        elif "follicleno.(l)" in c_lower or "follicleno(l)" in c_lower:
            rename_dict[c] = "Follicle No. (L)"
        elif "follicleno.(r)" in c_lower or "follicleno(r)" in c_lower:
            rename_dict[c] = "Follicle No. (R)"
        elif "amh" in c_lower and "ng/ml" in c_lower:
            rename_dict[c] = "AMH(ng/mL)"
        elif "fsh" in c_lower and "lh" not in c_lower and "miu/ml" in c_lower:
            rename_dict[c] = "FSH(mIU/mL)"
        elif "lh" in c_lower and "fsh" not in c_lower and "miu/ml" in c_lower:
            rename_dict[c] = "LH(mIU/mL)"
        elif "waist:hip" in c_lower or "whr" in c_lower:
            rename_dict[c] = "Waist:Hip Ratio"
        elif "endometrium" in c_lower:
            rename_dict[c] = "Endometrium (mm)"
        elif "cycle(r/i)" in c_lower:
            rename_dict[c] = "Cycle(R/I)"
        elif "cyclelength" in c_lower:
            rename_dict[c] = "Cycle length(days)"
    if rename_dict:
        df_clean = df_clean.rename(columns=rename_dict)
    return df_clean

def engineer_clinical_features(df: pd.DataFrame) -> pd.DataFrame:
    """
```

4. Module 3: Zero-Leakage Splitting & SMOTE-ENN (experiments/step2.py)

```
import os
import sys
import warnings
warnings.filterwarnings('ignore')

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')

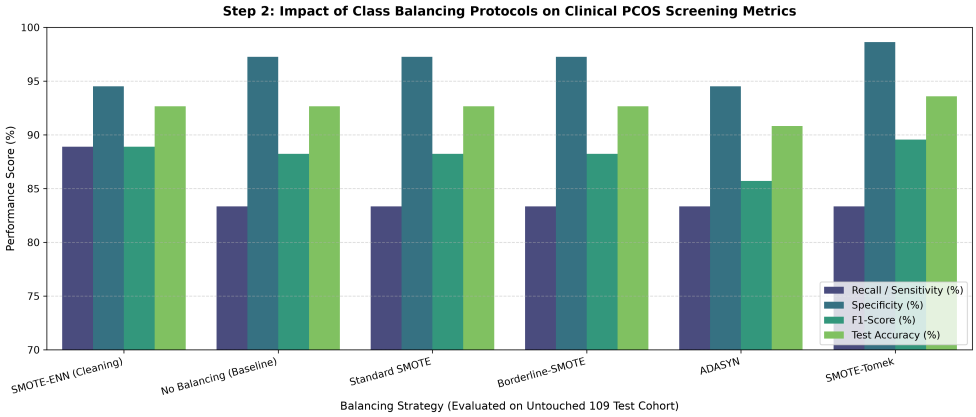
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import MinMaxScaler, RobustScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import ExtraTreesClassifier
import lightgbm as lgb
import xgboost as xgb
import catboost as cb

from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN
from imblearn.combine import SMOTEENN, SMOTETomek

from src.preprocessing import sanitize_clinical_dataframe
from src.feature_engineering import engineer_clinical_features

def run_splitting_and_balancing_benchmark(data_path: str = "data/PCOS_data.csv"):
    os.makedirs("outputs", exist_ok=True)
    print("=" * 80)
```

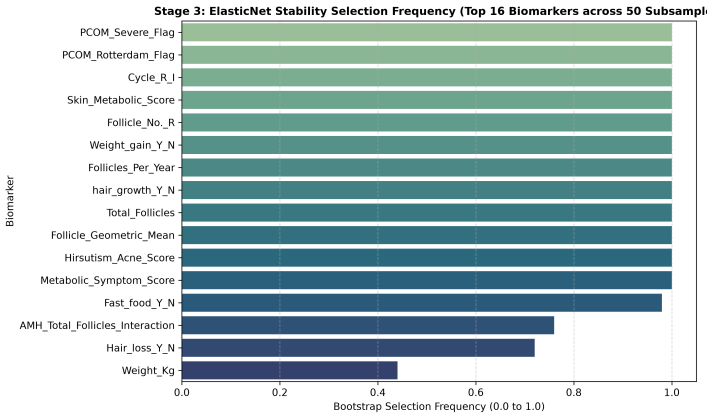


5. Module 4: Tri-Stage Hybrid Feature Selection (src/feature_selection.py)

```
import pandas as pd
import numpy as np
import re
from sklearn.feature_selection import SelectKBest, chi2, f_classif, RFE
from sklearn.linear_model import LogisticRegression, ElasticNet
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
from typing import List, Tuple, Dict, Any, Optional

# The 16 canonical features identified in earlier baseline literature
PAPER_16_FEATURES = [
    "Follicle No. (R)",
    "Follicle No. (L)",
    "hair growth(Y/N)",
    "Cycle(R/I)",
    "Weight gain(Y/N)",
    "Skin darkening (Y/N)",
    "Cycle length(days)",
    "Pimples(Y/N)",
    "Avg. F size (L) (mm)",
    "Marraige Status (Yrs)",
    "Avg. F size (R) (mm)",
    "RR (breaths/min)",
    "Fast food (Y/N)",
    "Weight (Kg)",
    "FSH(mIU/mL)",
    "AMH(ng/mL)"
]

def stage1_hybrid_filter(
    X_train: pd.DataFrame,
    y_train: pd.Series,
    target_k: int = 30
) -> Tuple[List[str], pd.DataFrame]:
```

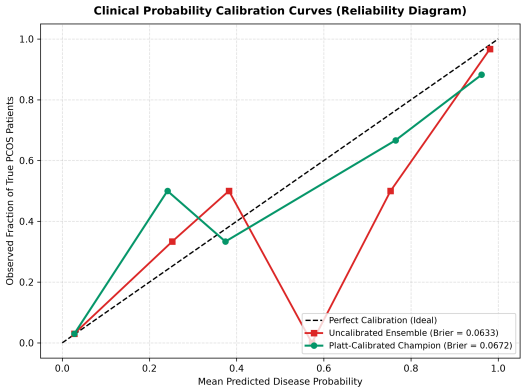


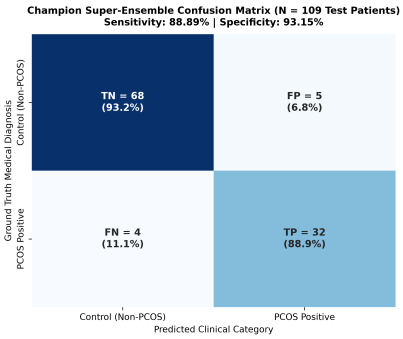
6. Module 5: Model Training, Super-Ensemble & Calibration (src/models.py)

```
import os
import re
import numpy as np
import pandas as pd
from typing import Dict, Any, Tuple, List, Optional

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier,
    ExtraTreesClassifier,
    GradientBoostingClassifier,
    AdaBoostClassifier,
    VotingClassifier,
    StackingClassifier
)
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    brier_score_loss
)
from scipy.stats import wilcoxon

import lightgbm as lgb
```





7. Module 6: 360° Explainable AI & DiCE Counterfactuals (src/xai.py)

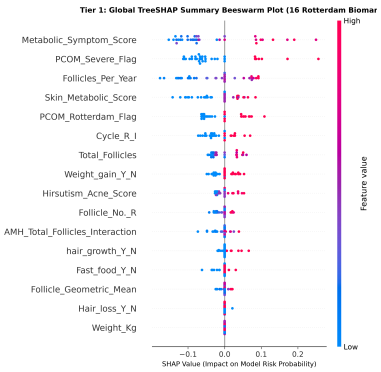
```
import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from typing import Dict, Any, Tuple, List, Optional

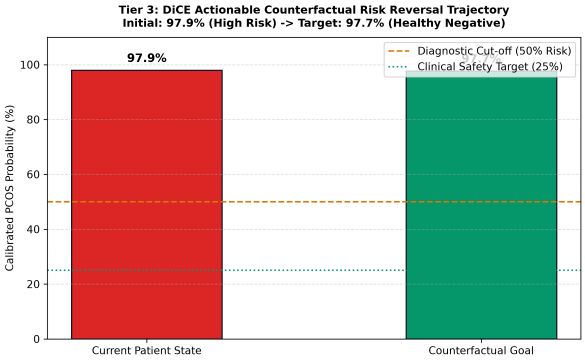
import shap
import lime
import lime.lime_tabular

def sanitize_col_name(c: str) -> str:
    return re.sub(r'[:,\(\)\[\]\{\}\"\/\\ ]+', '_', c).strip('_')

def compute_360_shap_explanations(
    champion_dict: Dict[str, Any],
    X_train: pd.DataFrame,
    X_test: pd.DataFrame,
    y_test: pd.Series,
    feature_names: List[str],
    output_dir: str = "outputs"
) -> Dict[str, Any]:
    """
    Tier 1 & Tier 2 Explainability:
    Computes Global SHAP Beeswarm, Feature Importance Bar, Interaction Dependence,
    and Local Patient-Specific Waterfall Plots for high-risk and low-risk cases.
    """
    os.makedirs(output_dir, exist_ok=True)
    print("\n" + "=" * 75, flush=True)
    print(" [XAI TIER 1 & 2] COMPUTING 360° SHAP POPULATION & PATIENT-SPECIFIC ATTRIBUTIONS", flush=True)
    print("=" * 75, flush=True)

    model = champion_dict["model"]
```





8. Module 7: Interactive CDSS Web Application (app.py)

```
import os
import sys
import re
import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
import joblib

# Set Page Config
st.set_page_config(
    page_title="PCOS-Prediction-XAI CDSS",
    page_icon="■",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for Medical / Clinical Aesthetic
st.markdown("""
<style>
    .main-header {
        font-size: 2.1rem;
        font-weight: 800;
        color: #0F172A;
        margin-bottom: 2px;
    }
    .sub-header {
        font-size: 1.05rem;
        color: #475569;
        margin-bottom: 20px;
    }
    .metric-card {
        background-color: #F8FAFC;
        border: 1px solid #E2E8F0;
        border-radius: 10px;
        padding: 16px;
        box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    }
    .risk-high {
        background-color: #FEF2F2;
        border-left: 5px solid #EF4444;
        padding: 14px;
        border-radius: 6px;
        color: #991B1B;
        font-weight: 600;
```

9. Module 8: Master Orchestration Pipeline (main.py)

```
import os
import sys
import glob
import pandas as pd
import numpy as np

from src.preprocessing import load_and_preprocess_data
from src.feature_selection import two_stage_feature_selection
from src.models import (
    get_base_models,
    get_champion_ensemble,
    evaluate_model,
    perform_10fold_cv
)
from src.xai import generate_shap_explanations, generate_lime_explanation

def create_synthetic_demo_data(file_path: str):
    """Creates synthetic placeholder dataset matching the PCOS schema if no data is found."""
    np.random.seed(42)
    n_samples = 541
    data = {
        "Sl. No": range(1, n_samples + 1),
        "Patient File No.": [f"PF_{i}" for i in range(1, n_samples + 1)],
        "PCOS (Y/N)": np.random.choice([0, 1], size=n_samples, p=[0.673, 0.327]),
        "Age (years)": np.random.randint(18, 45, size=n_samples),
        "Weight (Kg)": np.random.normal(60, 12, size=n_samples),
        "Height(Cm)": np.random.normal(160, 8, size=n_samples),
        "BMI": np.random.normal(24, 4, size=n_samples),
        "Blood Group": np.random.choice([11, 12, 13, 14, 15], size=n_samples),
        "Pulse rate(bpm)": np.random.randint(60, 100, size=n_samples),
        "RR (breaths/min)": np.random.randint(12, 25, size=n_samples),
        "Hb(g/dl)": np.random.normal(12.5, 1.5, size=n_samples),
        "Cycle(R/I)": np.random.choice([2, 4, 5], size=n_samples),
        "Cycle length(days)": np.random.randint(2, 12, size=n_samples),
        "Marraige Status (Yrs)": np.random.randint(0, 20, size=n_samples),
        "Pregnant(Y/N)": np.random.choice([0, 1], size=n_samples),
        "No. of aborptions": np.random.randint(0, 4, size=n_samples),
        "FSH(mIU/mL)": np.random.exponential(5, size=n_samples),
        "LH(mIU/mL)": np.random.exponential(4, size=n_samples),
        "FSH/LH": np.random.normal(1.5, 0.5, size=n_samples),
        "Hip(inch)": np.random.normal(38, 4, size=n_samples),
        "Waist(inch)": np.random.normal(32, 4, size=n_samples),
        "Waist:Hip Ratio": np.random.normal(0.85, 0.05, size=n_samples),
        "TSH (mIU/L)": np.random.normal(2.5, 1.0, size=n_samples),
        "AMH(ng/mL)": np.random.exponential(4.5, size=n_samples),
```